# Assignment 1, Smoothing

## A: Density Estimation

Carl-Emil Krarup Jensen

University of Copenhagen
Department of Mathematical Sciences

November 1, 2026

# Theoretical background

Goal: Perform a nonparametric density using the Epanechnikov kernel

$$K(x) = \frac{3}{4}(1 - x^2)\mathbf{1}_{[-1,1]}(x).$$

Let $X_1, \ldots, X_N$ be i.i.d, with distribution $f_0$ then the kernel density estimator is defined as

$$\hat{f}_h(x) = \frac{1}{hN} \sum_{i=1}^{N} K\left(\frac{x - x_i}{h}\right)$$

The quantity of interest for this project is the positive scalar $h > 0$.

# Theoretical background

The bandwidth $h$, controls the bias variance tradeoff in the density estimation:

$$\text{MISE}(h) = \int \mathbf{Var}(\hat{f}_h(x))dx + \int \text{bias}(\hat{f}_h(x))^2.$$

Let $||f_0''||_2^2 < \infty$ , the asymptotic mean integrated error (AMISE) is given by

$$\text{AMISE}(h) = \frac{||K||_2^2}{Nh} + \frac{h^4\sigma_K^4||f_0''||_2^2}{4}.$$

For the epanechnikov kernel we have

$$||K||_2^2 = \frac{9}{16}\int_{-1}^{1}(1-x^2)^2dx = \frac{3}{5} \text{ and } \sigma_K^2 = \frac{3}{4}\int_{-1}^{1}x^2(1-x^2)dx = \frac{1}{5}.$$

# Theoretical background

Solving $\text{AMISE}'(h) = 0$ gives

$$h_{\text{AMISE}} = \left( \frac{||K||_2^2}{N\sigma_K^4 ||f_0''||_2^2} \right)^{1/5}.$$

And since $\text{AMISE} \overset{h \to \infty}{\to} \infty$ and $\text{AMISE} \overset{h \to 0}{\to} \infty$, $h_{\text{AMISE}}$ is a unique global minimizer.
Problem: Only an oracle would know the true underlying density $f_0$.
Therefore the above bandwidth can be referred to as the oracle bandwidth.

# Theoretical background

In an attempt to discover insights of the oracle, we implement a pilot estimate $\tilde{f}$ of $f_0$. In particular

$$||\tilde{f}_0''||_2^2 = \frac{1}{N^2 r^6} \sum_{i=1}^{N} \sum_{j=1}^{N} \int H'' \left( \frac{x - x_i}{r} \right) H'' \left( \frac{x - x_j}{r} \right) dx.$$

We use the gaussian kernel for $H$, and Silverman's rule of thumb to choose $r = 0.9\tilde{\sigma} n^{-1/5}$.

# Theoretical background

Another way of choosing bandwidth selection is by cross validation:
The index set $\{1, 2, \ldots, N\}$ is randomly split into $k$ disjoint sets $H_1, H_2, \ldots H_k$ of roughly equal size $N/k$.
This defines $k$ rounds of data splitting, with $I(i) = \{1, 2, \ldots, N\} \setminus H_i$, and $J(i) = H_i$.
Set $N_i = |I(i)|$, then the kernel estimator for the partition indexed by $I(i)$ is

$$\hat{f}_h^{-i} = \frac{1}{hN_i} \sum_{j=1}^{N_i} K\left(\frac{x - x_j}{h}\right).$$

# Theoretical background

We summarize the assessment via two criteria:

- Log likelihood k fold

$$\ell_{cv}(h) = \sum_{i=1}^{k} \sum_{j \in H_i} \log(\hat{f}_h^{-i}(x_j))$$

- Least squares leave one out

$$LS_{cv}(h) = \int \hat{f}_h^{-i}(x_i)^2 dx - \frac{2}{n} \sum_{i=1}^{N} (\hat{f}_h^{-i}(x_i))$$

We then define the cross validation bandwidth selections as

$$h_{\ell_{cv}} = \underset{h}{\operatorname{argmax}}\, \ell_c v(h)$$

and

$$h_{ls_{cv}} = \underset{h}{\operatorname{argmin}}\, LS_{cv}(h)$$

```
cv_loglik_score <- function(
    ...
    for (j in seq_len(k)) {
        test <- x[fold_id == j]
        train <- x[fold_id != j]
        density <- kde_at(
          eval_points = test,
          x = train,
          h = h,
          kernel = kernel)
        ...
        score <- score + sum(log(density))

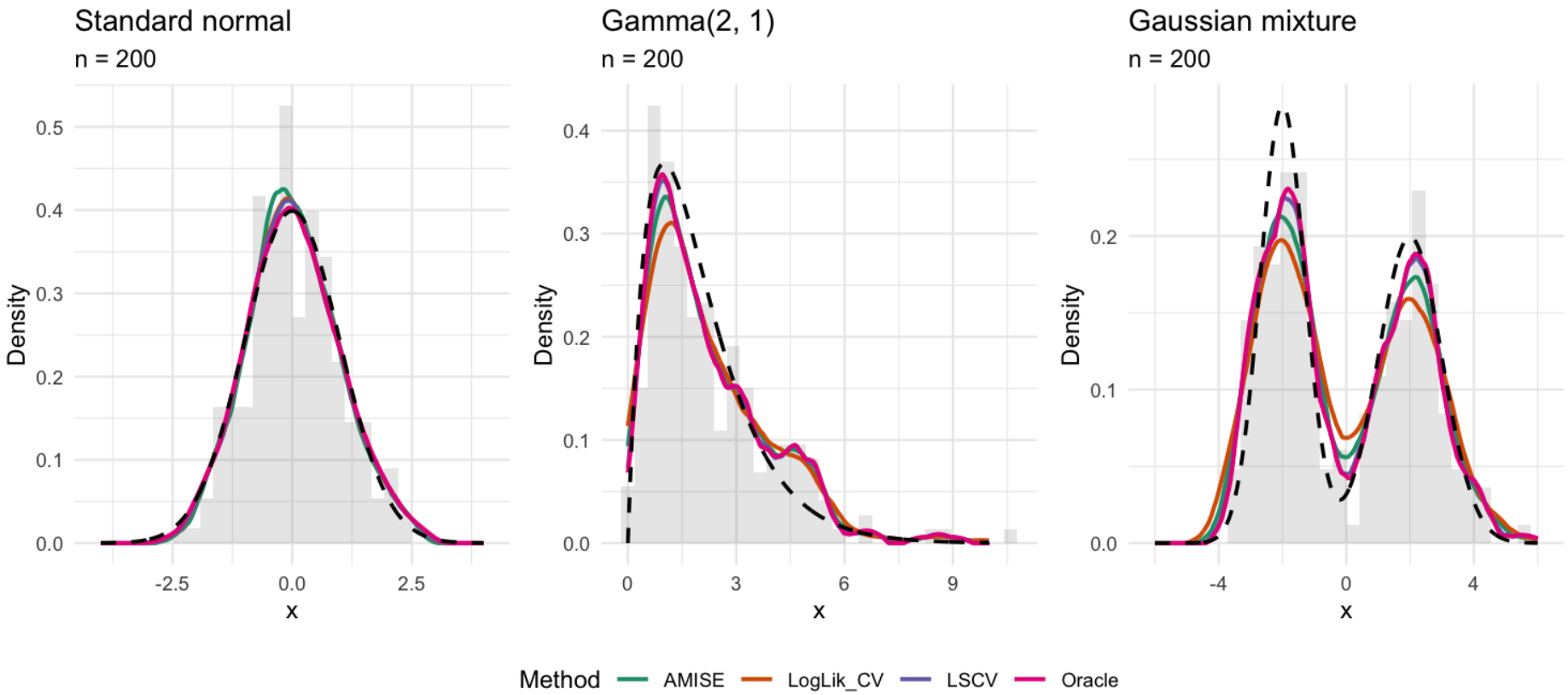
cv_ls_score <- function(
    ...
  for (j in seq_len(k)) {
    test <- x[fold_id == j]
    train <- x[fold_id != j]
    density_grid <- kde_at(...)
    integral_term <- sum(
      diff(grid) *
        (head(squared_density, -1) +
            tail(squared_density, -1)) / 2)
    density_test <- kde_at(eval_points = test,x = train,
    h = h, kernel = kernel)
    validation_term <- 2 * mean(density_test)
    fold_scores[j] <- integral_term - validation_term}
  mean(fold_scores)}
```

# Numerical comparisons

We simulate 200 pseudo random i.i.d draws from the

- Standard gaussian distribution
- Gamma(2,1) distribution
- Mixed gaussian distribution: $Z = \frac{1}{2}X + \frac{1}{2}Y$.
  $X \sim \mathcal{N}(-2, 0.49)$, $Y \sim (2, 1)$

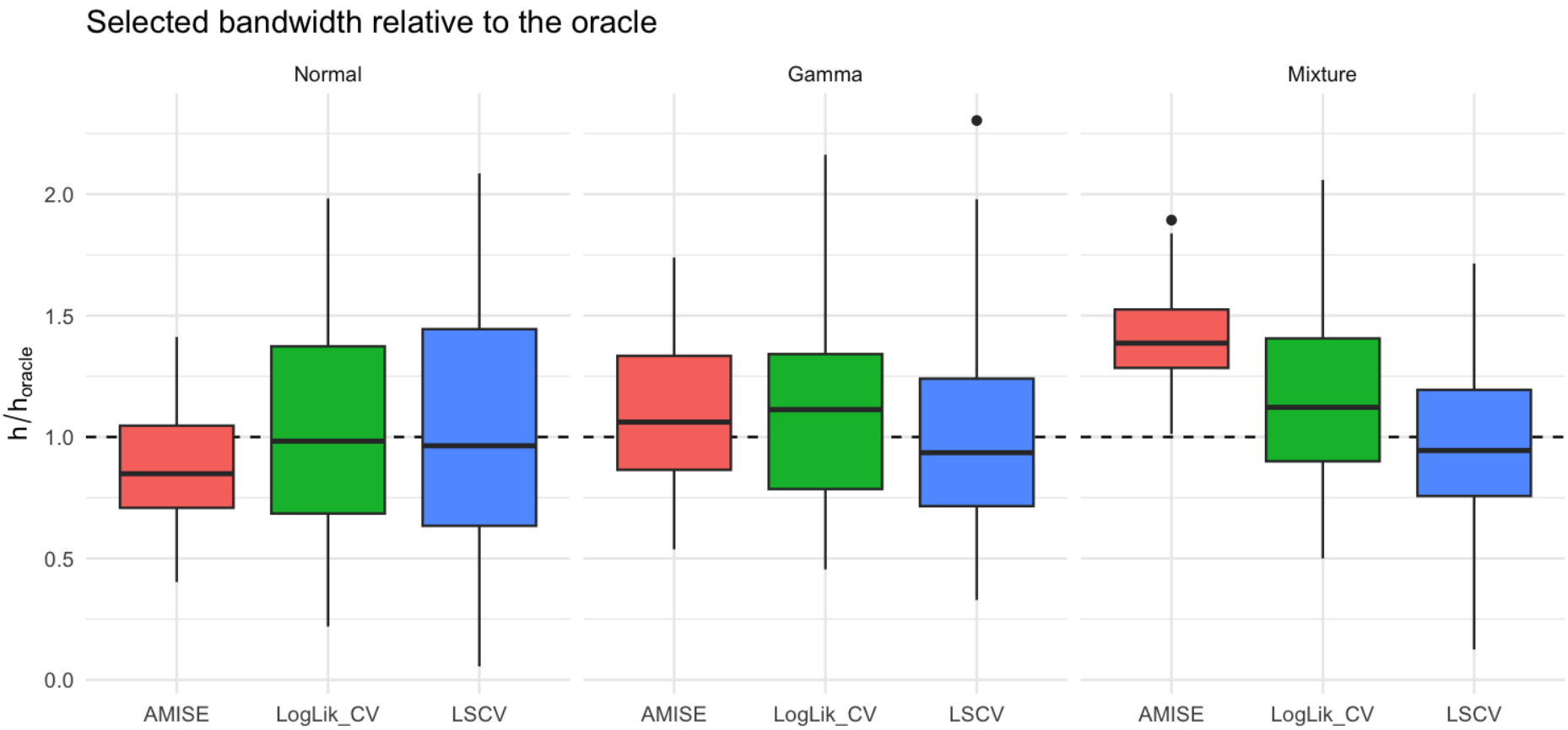
# Numerical comparisons



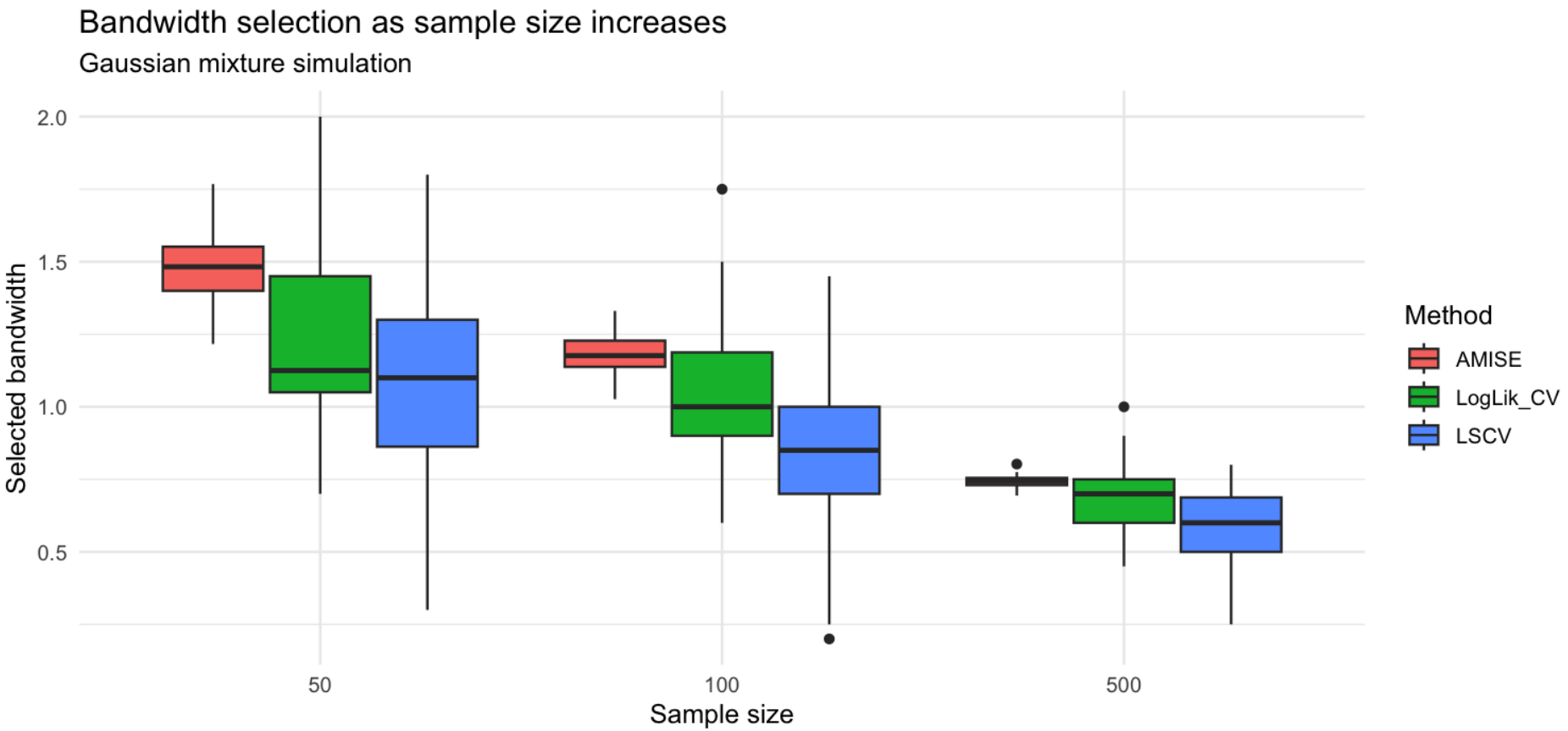
# Numerical comparisons

| **Scenario** | **Method** | **h** | **ISE** |
|---|---|---|---|
| Normal | AMISE | 0.642 | 0.00131 |
| | LogLik CV | 0.787 | 0.00069 |
| | **LSCV** | **0.812** | **0.00063** |
| | *Oracle* | 0.910 | 0.00053 |
| Gamma | AMISE | 0.823 | 0.00696 |
| | LogLik CV | 1.082 | 0.00934 |
| | **LSCV** | **0.664** | **0.00627** |
| | *Oracle* | 0.615 | 0.00620 |
| Mixture | AMISE | 1.052 | 0.00788 |
| | LogLik CV | 1.405 | 0.01260 |
| | **LSCV** | **0.777** | **0.00603** |
| | *Oracle* | 0.711 | 0.00593 |

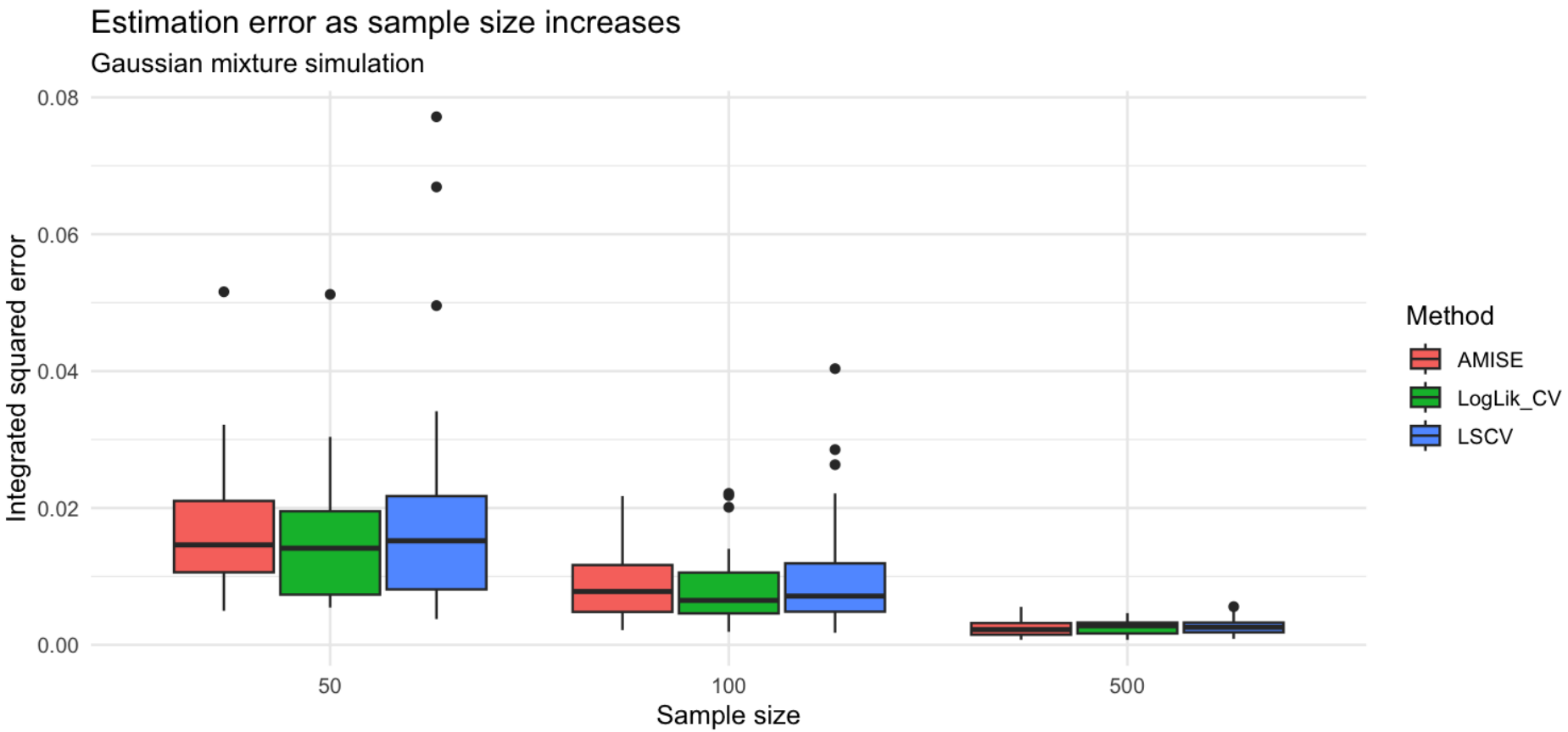
# Numerical comparisons



# Numerical comparisons



# Numerical comparisons



## Comparison with R's built in bandwidths

**Details**

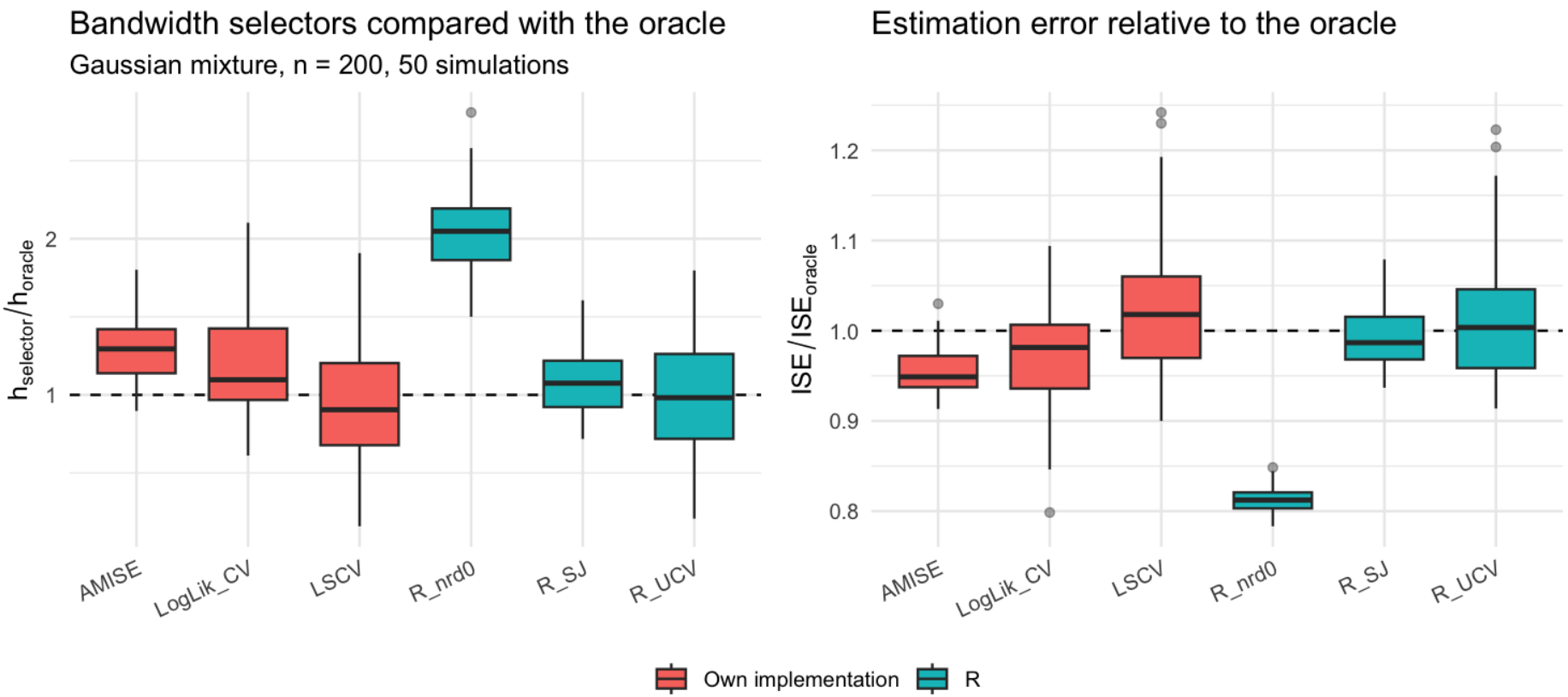
`bw.nrd0` implements a rule-of-thumb for choosing the bandwidth of a Gaussian kernel density estimator. It defaults to 0.9 times the minimum of the standard deviation and the interquartile range divided by 1.34 times the sample size to the negative one-fifth power (= Silverman's 'rule of thumb', Silverman (1986, equation 3.31 on page 48)) *unless* the quartiles coincide when a positive result will be guaranteed.

`bw.nrd` is the more common variation given by Scott (1992), using factor 1.06.

`bw.ucv` and `bw.bcv` implement unbiased and biased cross-validation respectively.

`bw.SJ` implements the methods of Sheather and Jones (1991) to select the bandwidth using pilot estimation of derivatives. The algorithm for method `"ste"` solves an equation (via `uniroot`) and because of that, enlarges the interval `c(lower, upper)` when the boundaries were not user-specified and do not bracket the root.

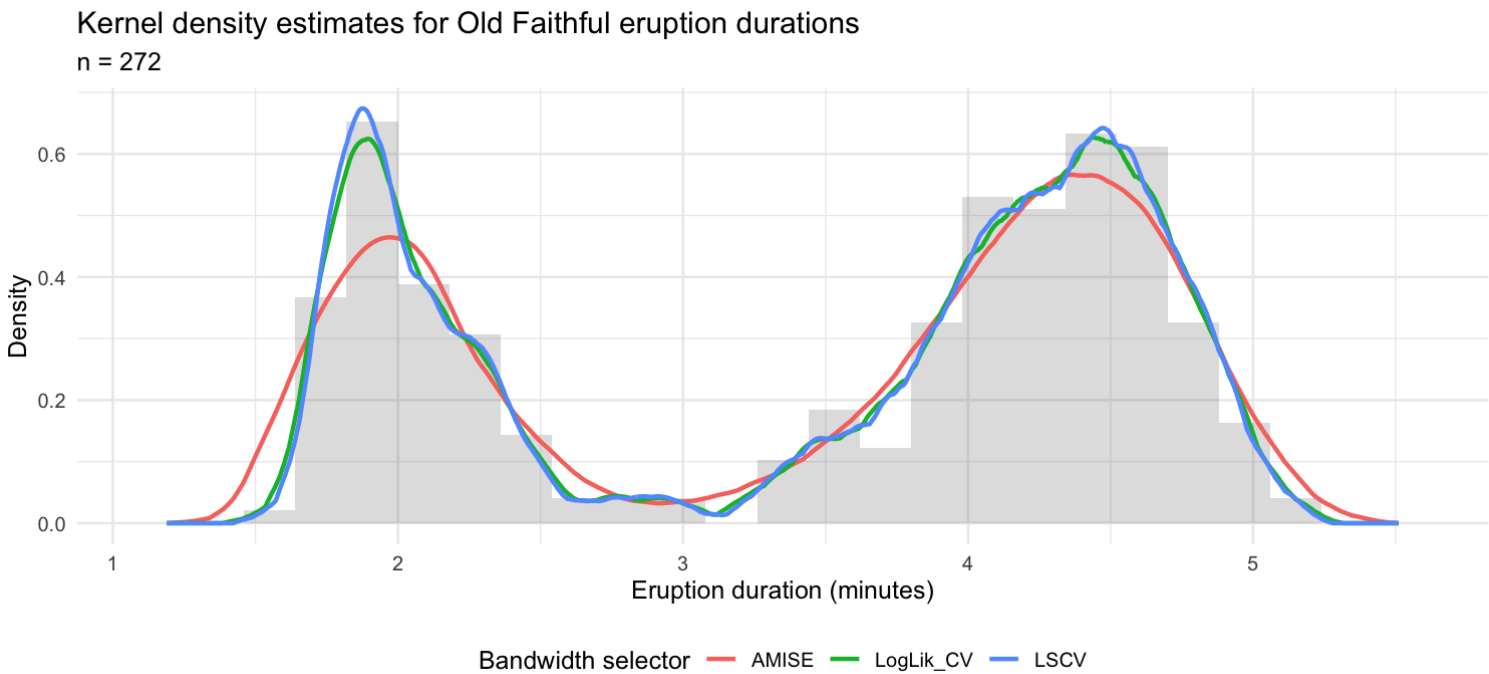
# Numerical comparisons



# Numerical comparisons

We compare the bandwidth selection methods with real data on eruption durations from the faithful dataset, with 272 observations.

| | **AMISE** | **LogLik CV** | **LSCV** |
|---|---|---|---|
| **h** | 0.411 | 0.215 | 0.178 |



# Implementation: Choices, tests and construction

All helper functions have been structured as follows.

## Project structure

```
assignment_1
├── project_A
│   ├── bandwidth_AMISE.R
│   ├── bandwidth_cv.R
│   ├── kernel_density_estimation.R
│   ├── kernels.R
│   ├── simulation_utils.R
│   └── utils.R
├── report
└── tests/testthat
    ├── test-amise.R
    ├── test-cv.R
    ├── test-kde.R
    └── test-kernel.R
```

## Example: `kernels.R`

```
epanechnikov_kernel <- function(u) {
  (3 / 4) * (1 - u^2) *
    (abs(u) <= 1)
}

RK_epanechnikov <- function() {
  3 / 5
}

mu2_epanechnikov <- function() {
  1 / 5
}
```

# Implementation: Choices, tests and construction

We already covered the mathematical theory underneath.

Instead of showing how it is implemented step by step. We can show that the implementation passes a list of sanity checks.

We can then discuss optimization of the implementation through the lens of a complexity analysis.

### Vectorized implementation from CSwR

```
kern_dens_vec<-function(x, h, n = 512){
  ...
  const<-(sqrt(2 * pi) * h * length(x))
  for (i in seq_along(xx)) {
    y[i]<-sum(exp(-(xx[i] - x)^2 /
    (2 * h^2))) / const}
```

### The heart of the code:

```
kde_at <- function(
    eval_points,
    x,
    h,
    kernel = epanechnikov_kernel
) {
  ...
  vapply(
    eval_points,
    function(z) {
      u <- (z - x) / h
      sum(kernel(u)) / (length(x) * h)
    },
  )
}
```

# Implementation: Choices, tests and construction

Test of the bandwidth selection logic

```
test_that("Epanechnikov kernel
integrates to one", {
  val <- integrate(
    epanechnikov_kernel,
    lower = -1,
    upper = 1
  )$value

  expect_equal(val, 1,
  tolerance = 1e-8)
})
```

Test of the kernel density estimator

```
    test_that("KDE approximately
    integrates to one", {
  fit <- kernel_density_estimator(
    x,
    h = 0.5,
    n = 5000
  )
  area <- sum(
    diff(fit$x) *
      (head(fit$y, -1) + tail(fit$y, -1))
  )
  expect_equal(
    area,
    1,
    tolerance = 1e-3
  )
})
```

# Benchmarking and profiling

We use R's built in profiling software `Rprof` The main bottlenecks of the code are

| **Component** | **Main bottleneck** | **Evidence** | **Cause** |
|---|---|---|---|
| KDE | Kernel evaluation | $\approx 100\%$ in `kde_at()` | $O(NM)$ |
| AMISE | Pairwise distances | 87% self-time in `outer()` | $O(N^2)$ |
| Likelihood CV | Repeated KDE | 88% in `kde_at()` | KDE over $h \times k$ |
| LSCV | Repeated KDE | 85% in `kde_at()` | KDE over candidate $h$ |

## Conclusion

The four procedures reduce to two computational bottlenecks:

- Repeated KDE evaluation
- Pairwise $N \times N$ computation in AMISE

# Benchmarking and profiling

We implement two alternative kernel density estimators:

Matrix Estimator

```
D <- outer(
  eval_points,
  x,
  "-"
) / h
K <- kernel(D)
rowSums(K) / (length(x) * h)
```

Binned estimator

```
  weights <- kde_bin_weights(
  x = x,
  lower = lower,
  upper = upper,
  B = B
)

D <- outer(
  eval_points,
  centers,
  "-"
) / h

K <- kernel(D) / h

drop(
  K %*% weights
)
```

# Benchmarking and profiling

The background of the binned estimator:
Replace the observations by $B$ equally spaced bin centers

$$c_j = L + (j-1)\Delta, \qquad \Delta = \frac{U-L}{B-1}, \qquad j = 1, \ldots, B,$$

where $[L, U]$ is the evaluation range. Each observation $X_i$ is assigned to its nearest center:

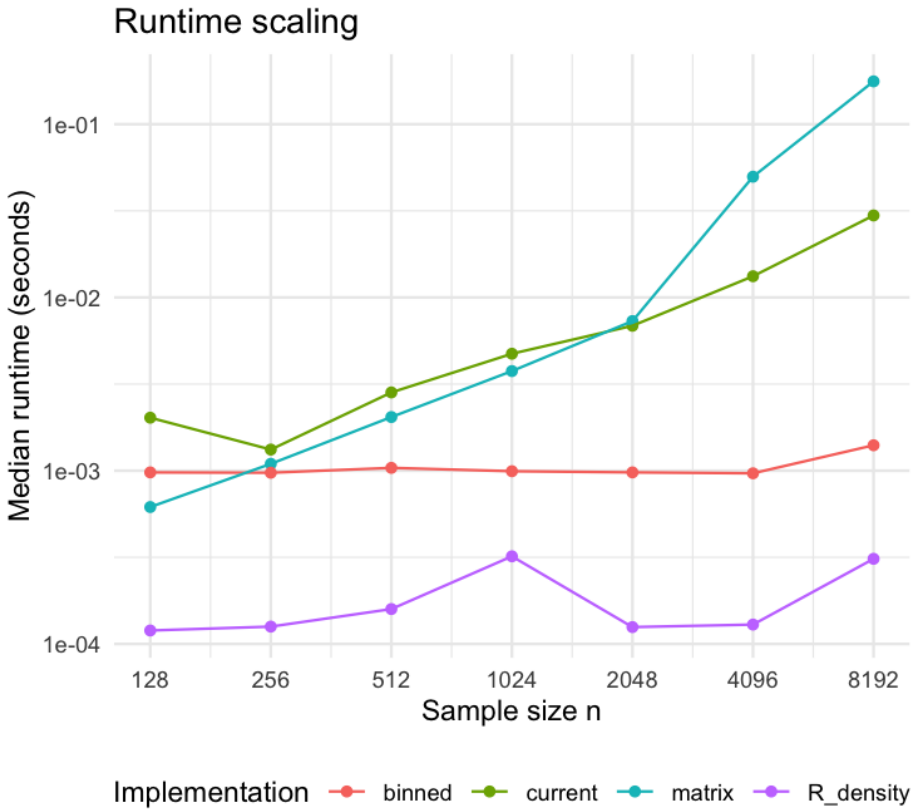
$$b(i) = 1 + \left\lfloor \frac{x_i - L}{\Delta} + \frac{1}{2} \right\rfloor.$$

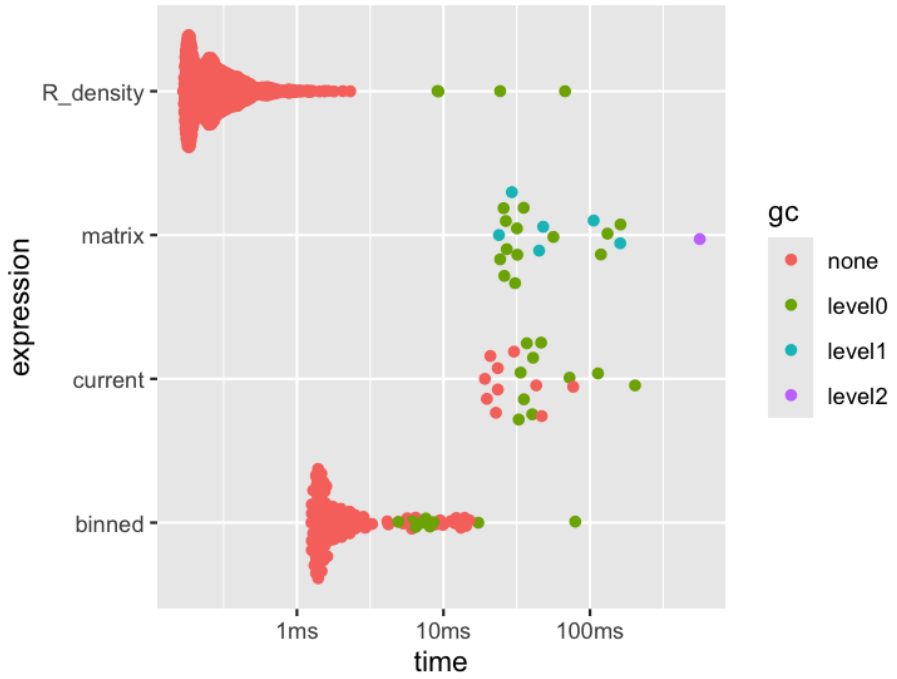
Let $N_j = \#\{i : b(i) = j\}$, and $w_j = \frac{N_j}{n}$. The binned approximation is

$$\tilde{f}_h(x) = \frac{1}{h} \sum_{j=1}^{B} w_j K\left(\frac{x - c_j}{h}\right)$$

# Benchmarking and profiling

Runtime comparison between kernel density estimators

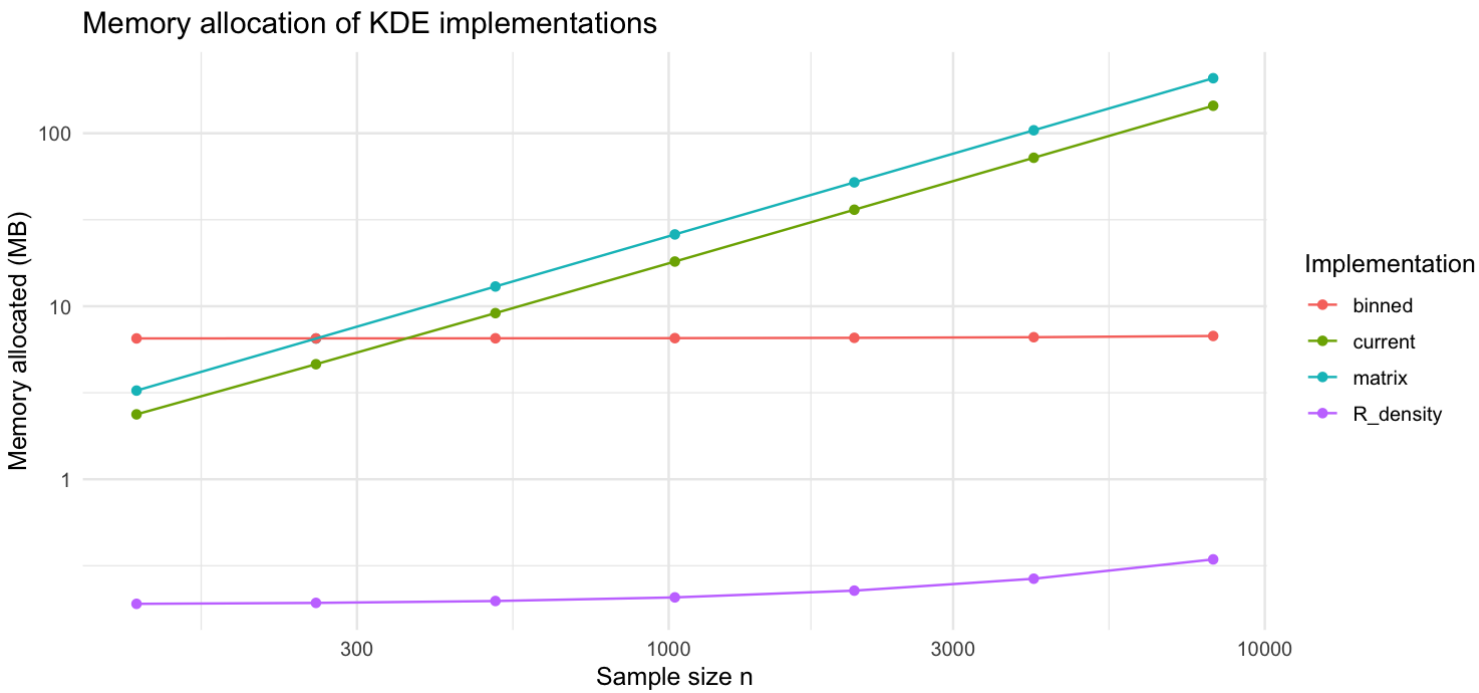



The beeswarm plot is produced with $n = 512$

# Benchmarking and profiling

Memory allocation of the kernel density estimators




The matrix implementation does not seem to improve in simulation studies of this size.
The binned estimator shows potential improvement of performance.
Does this however come at a cost?

# Discussion of choices and potential improvements

- The base KDE evaluates $N$ kernel contributions in $M$ output points and has complexity $O(MN)$.
- The matrix implementation creates an $M \times N$ matrix for each observation evaluation pair and performs matrix operations on this, thus, also having a complexity of $O(MN)$.
- The binned KDE first processes the data once, and then compares $m$ evaluation points with the $B$ centers, which has the complexity $O(N) + O(MB)$
- Caveat: as $B$ gets smaller approximation error will increase. $B$ controls the speed/accuracy tradeoff.
- `density()` disperses the mass of the distribution over a grid and uses fourier transformation convolution. [1]

Vectorization and algorithmic optimization are two distinct tools to improve complexity.

[1] See documentation https://www.stat.ethz.ch/CRAN/doc/manuals/r-patched/packages/stats/refman/stats.html?

# Discussion of choices and potential improvements

- A further analysis could look at a comparison between the kernel estimators depending on empirical skewness and kurtosis, and extending the numerical study to a broader class of distributions.
- An path of further research in the computational aspects of kernel density estimation is further exploration of algorithmic optimization.

# References I